

이벤트 시스템 — Stage 2 검증 결과 (effect + chain + choice)

날짜: 2026-05-01 결과:  PASS — 신규 28/28 + 회귀 1A 23 + 1A-2 25 + 1B 17 = 누적 93/93 + 스크린샷 4컷

선행: 이벤트 시스템 - 1B 검증 결과.md , 이제부터 할일.pdf

§1. 단계 정의

Stage 2 — **effect** + **chain (next)** + **choice** 통합 단계. 안 4 §9.1·§9.2·§9.4 한 번에. 검증 시나리오 = 노드 6 (수리) 진입 → 팔 수리 / 몸 수리 선택 → 효과 적용 → 완료 대사 → **chain** 종료.

설계 결정: - **EventManager** = **dependent sub-system** (RunManager.apply_event_action 직호출). 메모리 — 안 4 §0-E 그대로. - 노드 6 type "repair" (기존 **assault_trooper** 적 제거). - 5 개 신규 이벤트로 chain 구성: **repair_choice** → **repair_arm** / **repair_body** → **repair_done_arm** / **repair_done_body** .

§2. 산출물

신규 파일

- **test/test_event_2.{gd,tscn}** — main.tscn 인스턴스 + 시나리오 + 4 컷 스크린샷.

변경 파일

- **event_manager.gd**
- **begin_event** → **_begin_event_internal** (chain root 인자 받는 공용 핸들러) + **_transition_to_event** (chain 전이용 wrapper) 분리.
- kind 분기: **dialogue** / **effect** (신규) / **choice** (신규) / 미지원 스텝.
- **_dispatch_effect** — **_apply_event_actions** → **_finalize_event** 동기 진행.
- **_dispatch_choice** — **event_state** 채우고 입력 대기.
- **select_choice(idx)** — 가지의 **next** 로 chain 전이 (없으면 **_resolve_chain**).
- **_finalize_event** — **def.next** 검사. 있으면 **_transition_to_event** , 없으면 **_resolve_chain** .
- **_resolve_chain** — chain 끝, **event_resolved.emit({event_id: chain_root_id})** .
- LimboConsole: **event_choose <idx>** 추가.
- **run_manager.gd**

- **apply_event_action(action)** (신규) — Type 1 액션 디스패처. match 분기 → `_apply_<type>(params)`. EventManager 가 호출.
- `move_to_node` — type 분기 일반화. `target.type == "event"` 하드코드 제거 → research 외 모든 type 에 대해 `_resolve_event_for_node` 시도. (노드 6 type "repair" 가 잡히지 않던 버그 수정.)
- `game_data.gd`
- 노드 6: `enemy_id "assault_trooper"` 제거, `type: "repair"` 부여.
- EVENTS 에 5 개 신규: `repair_choice` (choice) / `repair_arm` (effect) / `repair_body` (effect) / `repair_done_arm` (dialogue) / `repair_done_body` (dialogue).
- `event_ui.gd`
- `_show_dialogue_mode` / `_show_choice_mode` 분리.
- choice kind 시 `HBoxContainer` 에 버튼 N 개 동적 생성, 클릭 → `select_choice(idx)`.
- 박스 클릭 advance 는 dialogue 모드 한정 (`mouse_filter` 토글).

§3. 시나리오 흐름

```

node 1
  ↓ run_start: intro_speech (dialogue) 자동 발화
  ↓ advance → 종료 → phase = map
node 1 → 5
  ↓ node_enter[type=event]: regression_speech (dialogue) 발화
  ↓ advance → 종료 → phase = map
node 5 → 6
  ↓ node_enter[type=repair]: repair_choice (choice) 발화
  ↓ select_choice(0) ← "팔 수리"
    chain → repair_arm (effect: arm_durability_boost +20)
    chain → repair_done_arm (dialogue: "팔 수리 완료.")
  ↓ advance → chain 종료 → phase = map

재진입 (5 → 6) — once_per: big_run 으로 미발화
end_internal_run cleared — seen_events 유지, 미발화

[reset + 재시작]
  ↓ select_choice(1) ← "몸 수리"
    chain → repair_body (effect: body_boost +20)
    chain → repair_done_body (dialogue)

```

§4. 검증 시나리오 (28 체크)

섹션 1·2 (사전 — 2)

- [x] intro 소비 후 phase = "map"
- [x] regression 소비 후 phase = "map"

섹션 3 — repair_choice 발화 (4)

- [x] phase = "event"
- [x] event_state.event_id = "repair_choice"
- [x] event_state.kind = "choice"
- [x] event_state.chain_root_id = "repair_choice"

섹션 4 — 팔 수리 chain 전이 + 효과 (6)

- [x] 선택 후 event_id = "repair_done_arm" (*effect → dialogue 까지 동기 전이*)
- [x] kind = "dialogue"
- [x] chain_root_id 유지 = "repair_choice"
- [x] phase = "event" 유지 (chain 진행 중)
- [x] arm hp += 20
- [x] arm max_hp += 20

섹션 5 — chain 종료 (5)

- [x] event_state = {}
- [x] phase = "map"
- [x] seen_events[repair_choice] = 1 (chain root 만)
- [x] seen_events[repair_arm] 미존재
- [x] seen_events[repair_done_arm] 미존재

섹션 6 — 재진입 once_per 필터 (2)

- [x] 재진입 시 phase = "map"
- [x] 재진입 시 event_state 비활성

섹션 7 — 회귀 후 once_per 유지 (3)

- [x] seen_events[repair_choice] 유지 = 1
- [x] 회귀 후에도 repair 미발화
- [x] 회귀 후에도 event_state 비활성

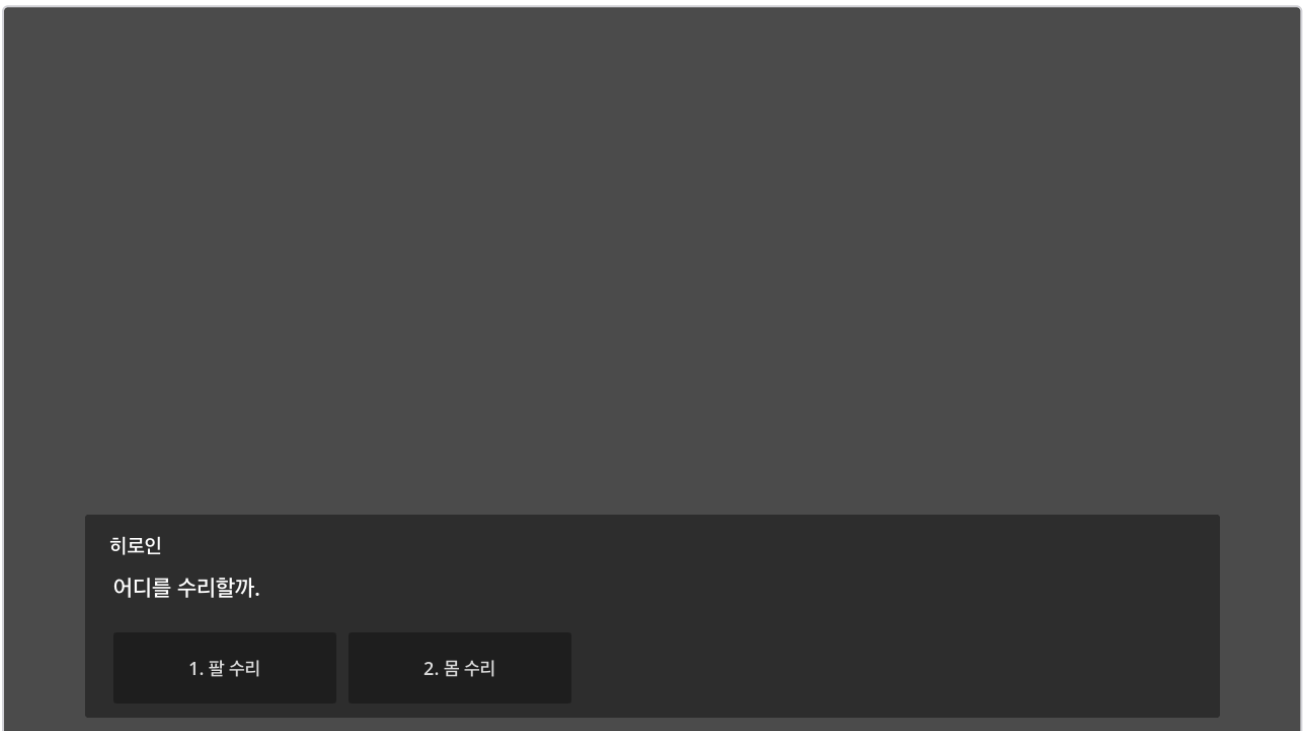
섹션 8 — reset + 몸 수리 가지 (6)

- [x] reset 후 repair_choice 다시 발화
- [x] 몸 가지 — event_id = "repair_done_body"

- [x] body_hp += 20
 - [x] body_max_hp += 20
 - [x] body 가지 chain 종료 → phase = "map"
 - [x] body 가지 — seen_events[repair_choice] = 1 (new big_run 카운트)
-

§5. 스크린샷

2_01 — repair_choice 발화 시점



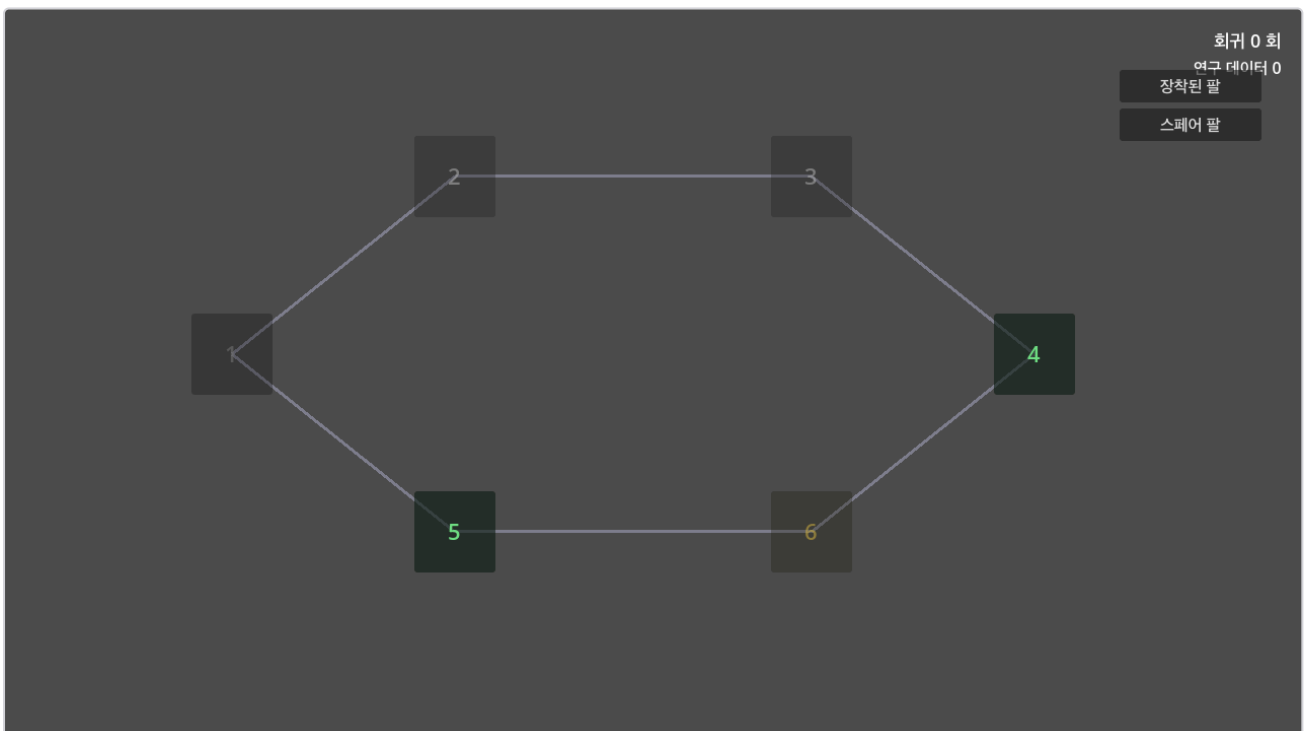
화자 "히로인" + prompt "어디를 수리할까." + 두 버튼 "1. 팔 수리" / "2. 몸 수리". 다른 UI 모두 숨김.

2_02 — 팔 수리 후 chain 종착점 (repair_done_arm dialogue)



`select_choice(0)` 호출 후 chain 이 effect (즉시 적용) → dialogue 까지 동기 전이. 화면에는 dialogue 만. arm hp / max_hp 는 이미 +20 적용됨.

2_03 — chain 종료 후 맵 복귀



advance_line 으로 chain 종료. event_screen 사라지고 맵 노출. 노드 6 visited (회색) 처리 반영. 다른 UI 정상.

2_04 — 몸 수리 가지 (reset 후 다른 분기)

히로인
몸 수리 완료.

▼ 클릭

같은 `repair_choice` 에서 `select_choice(1)` 선택 시 다른 `chain ( repair_body → repair_done_body )` 으로 분기. 결과 화면 = "몸 수리 완료." dialogue.

§6. 헤드리스 실행 명령

```
GODOT="/Users/js/Desktop/Godot.app/Contents/MacOS/Godot"
PROJ="/Users/js/Desktop/퍼즐던전 고도"

# 시뮬 회귀 (--headless OK)
"$GODOT" --headless --path "$PROJ" res://test/test_event_1a.tscn
"$GODOT" --headless --path "$PROJ" res://test/test_event_1a2.tscn

# UI + 스크린샷 (--headless 빼기)
"$GODOT" --path "$PROJ" res://test/test_event_1b.tscn
"$GODOT" --path "$PROJ" res://test/test_event_2.tscn
```

각 종료 코드 0 = PASS. PNG 는 `res://test_screenshots/` 에 저장.

§7. 자가점검

원칙 / 항목	상태
EventManager = dependent sub-system (RunManager.apply_event_action 직호출 OK)	✓

원칙 / 항목	상태
RunManager 단일 추적점 — seen_events ∈ big_run_data, chain_root_id 만 카운트	✓
Type 1 액션 단일 경로 — <code>apply_event_action</code> match → <code>_apply_<type></code> (escape 0)	✓
chain 진행 중 phase = "event" 유지 (boundary 안 빠짐)	✓
chain root 의 정의 = chain 의 첫 begin_event, transition 시 유지	✓
모르는 effect type → push_warning + 진행 (chain 안 깨짐)	(구조상 OK, 별도 테스트 추가 가능)
event_ui 자체 누적 상태 0 (kind 마다 from-scratch 렌더)	✓
한국어 폰트 헤드리스 렌더링 정상	✓

§8. 회귀 누적 — 93 체크

단계	체크 수	결과
Stage 1A	23	✓
Stage 1A-2	25	✓
Stage 1B	17	✓
Stage 2 (effect + chain + choice)	28	✓
합계	93	PASS

§9. 짚어들 — `move_to_node` 일반화

기존 코드는 `target.get("type") == "event"` 일 때만 EventManager 위임. 노드 6 type "repair" 가 잡히지 않아 첫 실행 시 section 8 fail. 수정 — research 외 모든 type 에 대해 `_resolve_event_for_node` 시도, 매치 없으면 fall-through. 이번 단계의 "find" 이고, 모든 미래 type ("event_combat" / "event_choice" 등) 도 자동 작동.

§10. 다음 단계

프로토타입 시스템 단은 사실상 완성. 안 4 §0 (A~H) 8항 모두 충족 + 4 kind 중 3 개 (dialogue / effect / choice) 실 구현 + chain 구현. movie kind 만 자산 의존으로 보류.

다음 갈래: 1. **컨텐츠 작성 (5천자 시나리오)** — 이제부터 할일 §2. 작가 진입점. 2. **§2 (long arc) 항목** — set_flag, 트리거 확장, once_per "internal_run" — 컨텐츠 작성하다 필요해지면 점진 도입. 3. **§3-1 변수 인터플레이션** — {run.body_hp} 같은 동적 텍스트. 4. **§3-3 movie kind** — 영상 자산 준비 후. 5. **소소한 코드 청소** — 직전 리뷰의 A/B/C/D/E (조회자 메서드, naming 정리 등). 컨텐츠 작성 흐름과 묶어 진행 가능.

GO 신호 + 어느 갈래 진입할지 알려주세요.